

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



BACHELOR THESIS

By

Pham Hoang An - 23BI14002

Title:

FEW-SHOT OPEN-SET KEYWORD SPOTTING AT VOCABULARY SCALE

A Metric-Learning Study of Feature Front-Ends, Encoders,
and Open-Set Rejection

Supervisor: Prof. Dr. Tran Hoang Tung

Hanoi, June 2026

To whom it may concern,

I, Dr. Tran Hoang Tung, certify that the thesis / internship report of Mr. Pham Hoang An is qualified to be presented in the Internship Jury 2025–2026.

Hanoi, June 2026

Supervisor's signature

Acknowledgements

First and foremost, I would like to express my deep gratitude to my supervisor, Dr. Tran Hoang Tung, for his insightful feedback and dedicated guidance throughout the three months of the internship. I would also like to sincerely thank Dr. Tran Giang Son for granting full access to the ICTLab server, which made the computational part of this project possible.

I am grateful to my family for their constant encouragement and support, which kept me motivated to complete this work. Finally, I would like to thank everyone who took the time to read and give feedback on this thesis.

Abstract

We study few-shot open-set keyword spotting (KWS): a single embedding encoder is trained once on a large word vocabulary and is then deployed to recognize an arbitrary user-defined keyword set from only a handful of enrollment examples, while rejecting out-of-vocabulary speech and silence. We build a fully reproducible pipeline around two compact encoders — a depthwise-separable CNN (DSCNN-L, ≈ 0.41 M parameters) and a temporal-attention edge model (EdgeSpot-Full $\tau=4$, ≈ 0.13 M parameters) — combined with three feature front-ends (MFCC, log-mel, and a trainable per-channel PCEN) and four metric-learning objectives (Triplet, Sub-center ArcFace (SCAF), Generalized End-to-End (GE2E), and a cosine knowledge-distillation (KD) objective from a frozen Wav2Vec 2.0 teacher). All systems are trained on Multilingual Spoken Words Corpus (MSWC) English and evaluated cross-corpus on Google Speech Commands v2 (GSC) under a strict EdgeSpot-exact 10-shot open-set protocol with 100 randomized runs. Through a 16-configuration factorial screen we show that (i) the trainable PCEN front-end is the single most reliable improvement (up to +9.2 accuracy points at 1% false-accept rate for the edge encoder); (ii) the centroid-based GE2E objective dominates classification-style losses for open-set generalization; (iii) crucially, the classification objective SCAF collapses when the training vocabulary grows to 37,387 classes unless its loss weight is reduced by four orders of magnitude. We further characterize how accuracy saturates near two million training clips and show that the cosine-KD objective reaches GE2E-level accuracy with roughly half the data. The best overall configuration (DSCNN-L + PCEN + GE2E) attains 86.36% open-set accuracy at 1% false-accept rate and 95.21 AUC on GSC test, while the best compact model (EdgeSpot-Full + PCEN + GE2E) reaches 82.87% at one third of the parameters. We release the complete code, training/evaluation protocols, and an offline web demonstrator.

Keywords: keyword spotting, few-shot learning, open-set recognition, metric learning, PCEN, GE2E, MSWC, Google Speech Commands.

Contents

Acknowledgements	i
Abstract	ii
1 Introduction	1
1.1 Context and Motivation	1
1.2 Project objectives	2
1.3 Desired outcomes	3
1.4 The problem in plain terms	3
1.5 Thesis structure	4
2 Background and Related Work	5
2.1 From sound to data	5
2.2 Time–frequency representations: the mel-spectrogram	6
2.3 Keyword spotting: closed-set vs. open-set	8
2.4 Few-shot learning and metric learning	9
2.5 Angular-margin classification and open-set rejection	10
3 Proposed Method	11
3.1 Problem formulation	11
3.2 Datasets and Training Protocol	13
3.3 System Architecture	15
3.3.1 Feature front-ends	16
3.3.2 Encoders	17
3.3.3 Metric-learning objectives	18
3.4 Open-set rejection heads	20
3.5 Evaluation methodology	20
3.6 Experimental design: what we measure	21
3.7 Deployment: streaming and demonstrator	21
4 Results and Analysis	23
4.1 Training dynamics	23

4.2	Q1: Which design axis matters	24
4.3	Q2: Scale — saturation and the SCAF collapse	27
4.4	Q3: Knowledge distillation trades teacher knowledge for data	29
4.5	Q4: Best deployable systems	29
4.6	Inference on long audio	31
5	Conclusion	32
5.1	Current status	32
5.2	What is good, what is weak	33
5.3	Future work	33

Chapter 1

Introduction

This chapter introduces the problem this thesis addresses and why it matters. Section 1.1 places keyword spotting in the context of modern voice interfaces and motivates the move from a closed-set design to a few-shot open-set one. Section 1.2 states the objectives of the thesis and Section 1.3 the expected outcomes. Section 1.4 explains the problem in plain terms. Finally, Section 1.5 outlines the structure of the whole thesis.

Before going into detail, Figure 1.1 summarizes the system at a conceptual level: the encoder is *trained once* to learn how to compare two sounds; at deployment the user *enrolls* a few examples per keyword to form representative points (prototypes); each incoming sound is *matched* to the nearest prototype; and if it lies far from every prototype it is *rejected* as “not a keyword”. The entire thesis revolves around these four steps.

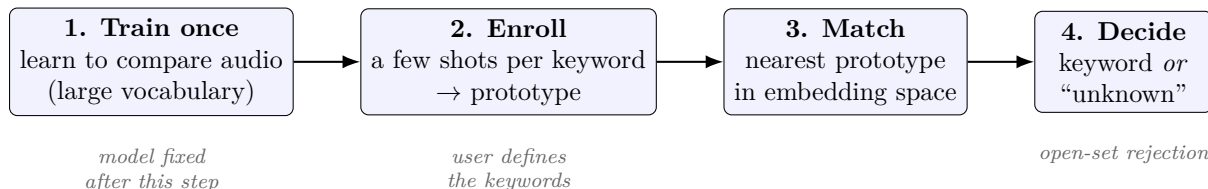


Figure 1.1: Conceptual overview of the few-shot open-set keyword-spotting system: train once → enroll → nearest-prototype match → open-set decision. The detailed technical diagram is given in Figure 3.1.

1.1 Context and Motivation

Keyword spotting (KWS) — the task of detecting a small set of spoken commands in a continuous audio stream — is the always-on entry point of nearly every modern voice interface. Classical KWS systems are trained as closed-set classifiers over a fixed vocabulary (e.g., the ten Google Speech Commands words). This design has two structural limitations. First, the keyword set is frozen at training time: supporting a new wake word or command requires collecting data and retraining the model. Second, a softmax classifier is, by construction, not open-set: every input is forced into one of the known classes, so unseen

words and background noise are misclassified as commands, producing false activations. This work targets the more realistic and more difficult few-shot open-set formulation. We train a single embedding encoder on a very large and diverse word vocabulary, and at deployment time the end user defines a keyword set by providing only a few (k) enrollment utterances per word. Each keyword is represented by a prototype (the mean embedding of its enrollment shots); recognition is performed by nearest-prototype matching in embedding space, and an explicit open-set decision rejects anything that is too far from all prototypes. Because the encoder is never retrained for a particular keyword, the same model serves unlimited downstream vocabularies; because rejection is distance-based, the system can abstain on out-of-vocabulary speech and silence. The question we pursue throughout this thesis is what makes such an encoder generalize and at what scale of the training vocabulary the problem must be tackled — whether a small vocabulary suffices or a large one is required, and if so how large — in order to achieve that generalization.

Contributions. Our main contributions are:

- A reproducible, dependency-light few-shot open-set KWS framework with two compact encoders, three feature front-ends, four metric objectives, and four open-set rejection heads (nearest-class-mean / OpenMax / energy / calibrated), all sharing a common prototype interface (Chapter 3).
- A 16-configuration factorial screen on full-vocabulary MSWC that isolates the contribution of each axis with paired deltas; PCEN and GE2E emerge as the two robust wins (Chapter 4).
- A scale analysis showing:
 - Accuracy saturation near ~ 2 M training clips; and
 - The failure mode of Sub-center ArcFace at 37k classes, with a gradient-magnitude explanation and a practical fix (Chapter 4).
- A knowledge-distillation study showing that a cosine objective against a frozen Wav2Vec 2.0 teacher matches GE2E accuracy with roughly half the training data (Chapter 4).
- A complete cross-corpus evaluation under a strict EdgeSpot-exact 10-shot protocol (100 runs), together with a CPU streaming pipeline (Silero VAD + smoothing state machine) and an offline web demonstrator (Chapter 3).

1.2 Project objectives

The primary objective of this thesis is to design, implement, and rigorously evaluate a *retraining-free, open-set* keyword-spotting system that learns a single transferable embed-

ding and supports arbitrary user-defined vocabularies from a few enrollment examples. Concretely, we aim to:

1. Build a compact encoder that runs offline on modest hardware;
2. Identify, through a controlled factorial study, which feature front-end, encoder backbone, and metric-learning objective generalize best across corpora;
3. Characterize how design choices behave as the training vocabulary scales from tens to tens of thousands of words; and
4. Deliver an honest open-set evaluation (DET/AUC/EER/ACC@FAR) under a strict, reproducible protocol, plus a working streaming demonstrator.

1.3 Desired outcomes

The system is expected to (i) recognize a user’s own keyword set enrolled from ≤ 10 examples per word without any retraining; (ii) reject out-of-vocabulary speech and silence with a controllable false-accept rate; (iii) generalize across recording conditions, demonstrated by training on one corpus (MSWC) and testing on a different one (GSC); and (iv) remain small enough for offline, on-device use. The accompanying deliverables are the trained encoders, the evaluation/training scripts, and an offline web demonstrator with long-audio timing analysis.

1.4 The problem in plain terms

Keyword spotting is the always-on software component of voice-controlled devices (smart speakers, earbuds) whose task is to detect a small number of short spoken commands such as “stop” or “next”, or a wake word. Because it must run continuously, it is deliberately kept small and energy-efficient; the full, resource-intensive speech recognizer is activated only *after* a keyword has been detected. Keyword spotting therefore acts as the gateway to most voice interfaces.

Limitations of the closed-set approach. Most commercial keyword-spotting systems are trained as classifiers over a *fixed* list of words (for example, the ten commands *yes*, *no*, *up*, *down*, ...). This design has two limitations. First, the keyword set is fixed at training time: adding a new command or word requires the manufacturer to collect data and retrain the model. Second, a closed-set classifier is forced to choose a label for *every* input, including a cough, music, or a previously unseen word; the device is therefore prone to *false activations* when no command was actually spoken.

The approach taken in this thesis. This thesis builds the system around *metric learning*: instead of memorizing a fixed set of labels, the model learns the more general skill of deciding whether two audio segments contain the same word. This skill is trained once on a large and diverse corpus of spoken words. At deployment, the user defines a command set by providing a few recordings per word, and the system requires no retraining. In terms of representation, each command is summarized as a single representative point in a feature space (an *embedding*); to recognize an input, the system maps the new audio into this space and determines which command it lies closest to. If the audio lies far from all enrolled commands, the system concludes “not a keyword” — this is the *open-set* capability that lets it ignore irrelevant speech and noise.

Practical value and challenges. This approach offers three benefits: (i) personalization without retraining, which suits privacy-preserving on-device assistants, accessibility tools, and customizable industrial controls; (ii) fewer false activations thanks to the rejection capability; and (iii) a small model footprint ($\sim 0.1\text{--}0.4$ million parameters), sufficient to run offline on-device and thereby preserve privacy. At the same time, the problem poses three main challenges. The first is *generalization*: the model must operate on words, recording devices, and accents unseen during training, so this thesis deliberately trains on one corpus and evaluates on a different one. The second is the *open-set boundary*: reliably rejecting the very large space of “everything that is not a command” is considerably harder than a few known commands, and the trade-off between over-rejection and under-rejection must be quantified honestly. The third is *scale*: as the training vocabulary grows from a few dozen to nearly forty thousand words, some techniques that are effective at small scale break down — a phenomenon analyzed in Chapter 4.

1.5 Thesis structure

The remainder of this thesis is organized as follows. Chapter 2 presents the necessary background: how sound is digitized and represented as a mel-spectrogram, the difference between closed-set and open-set recognition, the ideas of few-shot and metric learning, and the related work this thesis builds upon. Chapter 3 describes the proposed method: the problem formulation, the feature front-ends, the two encoders, the metric-learning objectives, the open-set rejection heads, the evaluation methodology, and the experimental design — this chapter states *what was done and measured*. Chapter 4 presents and analyzes the results around four research questions, tied to practical use. Chapter 5 summarizes the current status, strengths, weaknesses, and future directions.

Chapter 2

Background and Related Work

This chapter provides the background needed to follow the method. Section 2.1 describes how sound is digitized into data a computer can process; Section 2.2 presents the time–frequency representation (the mel-spectrogram) used as model input. Section 2.3 distinguishes closed-set from open-set keyword spotting, Section 2.4 introduces few-shot and metric learning, and Section 2.5 reviews the angular-margin and open-set-rejection methods that this thesis builds on and compares against.

2.1 From sound to data

Sound in nature is a *continuous pressure wave* that varies over time. A computer cannot store a continuous signal, so the first step is *digitization* via two operations: *sampling*, which measures the wave amplitude at evenly spaced instants, and *quantization*, which rounds each measured value to one of a finite set of levels. The result is a discrete sequence of numbers representing the waveform.

The sampling rate determines which frequency band the digital signal preserves. By the *Nyquist sampling theorem*, a sampling rate f_s faithfully represents only frequency components below $f_s/2$. The energy that distinguishes phonemes in speech is concentrated below about 8 kHz, so a rate of $f_s = 16,000$ samples per second (16 kHz) — with a Nyquist limit of 8 kHz — is sufficient and has become the de-facto standard for speech tasks. Increasing the sampling rate further mainly raises the computational cost while adding little to word-recognition quality. Throughout this thesis we use 16 kHz, single-channel (mono) audio.

So that every clip has the same input size, each spoken-command segment is standardized to exactly one second, i.e. a vector of 16,000 numbers (the *raw waveform*): shorter clips are zero-padded at both ends and longer clips are trimmed. Fixing the length lets us pack data into batches and keeps the downstream front-end at a constant tensor size. Figure 2.1(a) shows a one-second waveform of the word “yes”.

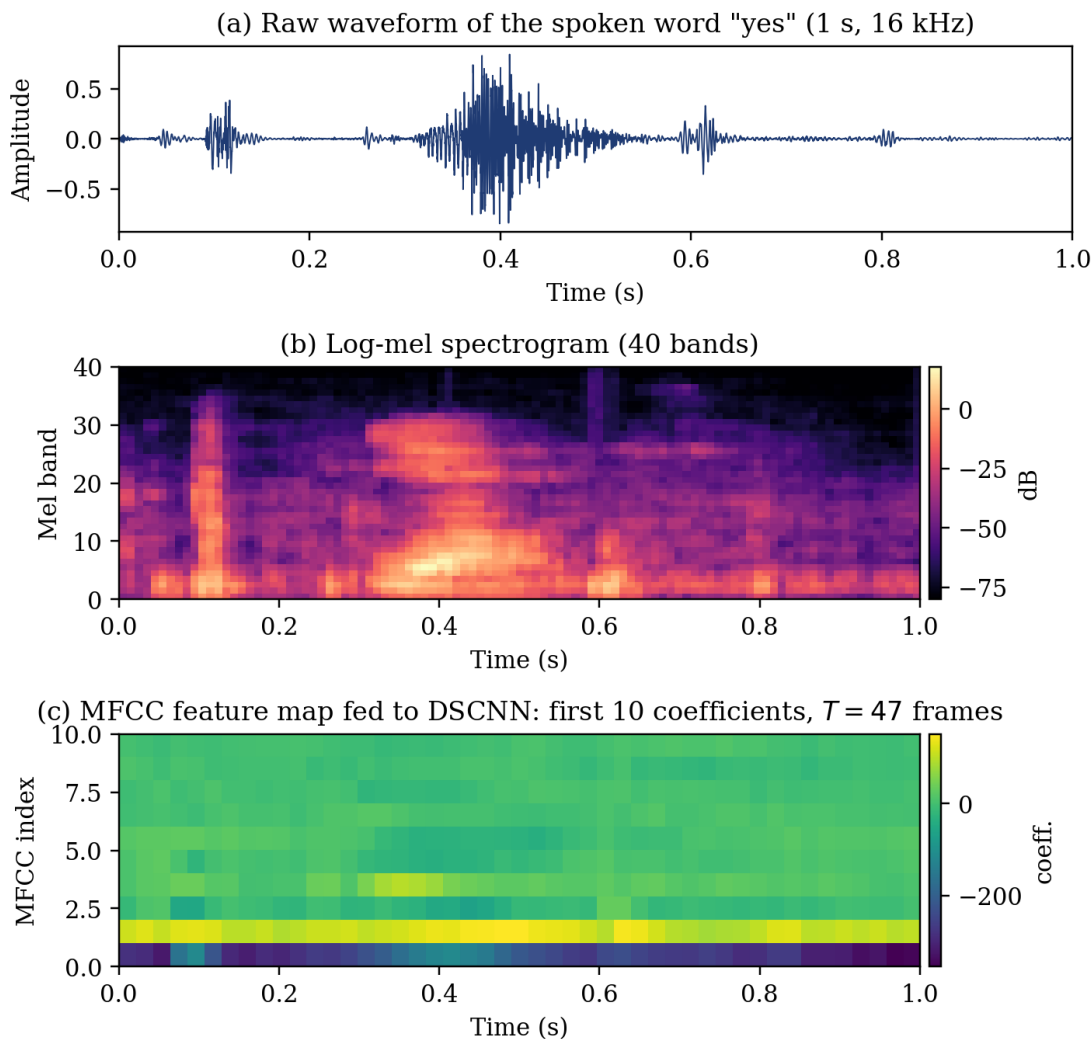


Figure 2.1: From sound to features, computed from a real Google Speech Commands recording of the word “yes”. (a) the 1 s, 16 kHz raw waveform; (b) the 40-band log-mel spectrogram (perceptual time–frequency image); (c) the first 10 MFCC coefficients over $T=47$ frames — the tensor actually fed to the DSCNN encoder.

Although it represents the signal, the raw waveform is a poor input for a classifier. First, it is *very high-dimensional* (16,000 values for one second). Second, it is *not invariant to harmless transformations*: the same word spoken slightly louder, shifted in time, or recorded on a different device produces a very different sample sequence at the per-sample level, even though a human still hears the same word. We therefore transform the waveform into a more compact and stable time–frequency representation, presented in Section 2.2.

2.2 Time–frequency representations: the mel-spectrogram

Speech is far easier to model in the *time–frequency* domain. We slide a short analysis window (here 25–40 ms) across the waveform in small hops (10–20 ms) and, for each window, compute the short-time Fourier transform (STFT) to obtain the energy at each frequency.

Two perceptual facts about human hearing then guide the next step: we perceive pitch roughly on a logarithmic scale, and we are more sensitive to differences at low frequencies than at high ones. The *mel scale* encodes exactly this: it warps linear frequency (Hz) into a perceptual “mel” axis via

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right), \quad (2.1)$$

where f is the frequency in Hz and m the corresponding value on the mel scale; the constants 700 and 2595 shape the curve so that the mel axis is approximately linear in Hz at low frequencies and compresses logarithmically at high frequencies, mirroring the decreasing resolution of human hearing. Consequently, a bank of (here 40) overlapping triangular filters spaced evenly on this mel axis pools the STFT energy into 40 *mel bands*, allocating more bands to the frequency region where human hearing is most sensitive. Taking a logarithm of the band energies (because loudness is also perceived logarithmically) yields the *log-mel spectrogram* of Figure 2.1(b): a compact image in which the horizontal axis is time, the vertical axis is mel frequency, and brightness is energy.

MFCC. A further classical step decorrelates the log-mel bands with a discrete cosine transform (DCT), producing *Mel-Frequency Cepstral Coefficients* (MFCCs). The low-order coefficients capture the slowly varying spectral envelope (which carries most of the phonetic identity), so it is common — and we follow this — to keep only the first few coefficients. Figure 2.1(c) shows the 10-coefficient MFCC map we feed to one of our encoders. The whole front-end is therefore a small, fixed transformation that turns one second of audio into a single-channel “image” of shape time×features, on which a convolutional network can operate.

PCEN. The static logarithm used above is simple but not adaptive to recording conditions. *Per-Channel Energy Normalization* (PCEN) [10] replaces it with a *trainable*, per-band automatic-gain-control followed by a soft compression; its parameters are learned jointly with the network. PCEN is designed for robust, far-field detection and, as Chapter 4 shows, is the single most reliable improvement in our study. The exact equation is given in Section 3.3.1.

To build intuition for what the model actually sees, Figure 2.2 shows the log-mel spectrograms of six different command words. Each word has a characteristic time–frequency signature — the rising voiced segment of “yes”, the plosive burst of “stop”, the nasal onset of “no” — and it is this structure, not the raw waveform, that the encoder learns to compare.

Log-mel spectrograms of six Google Speech Commands words

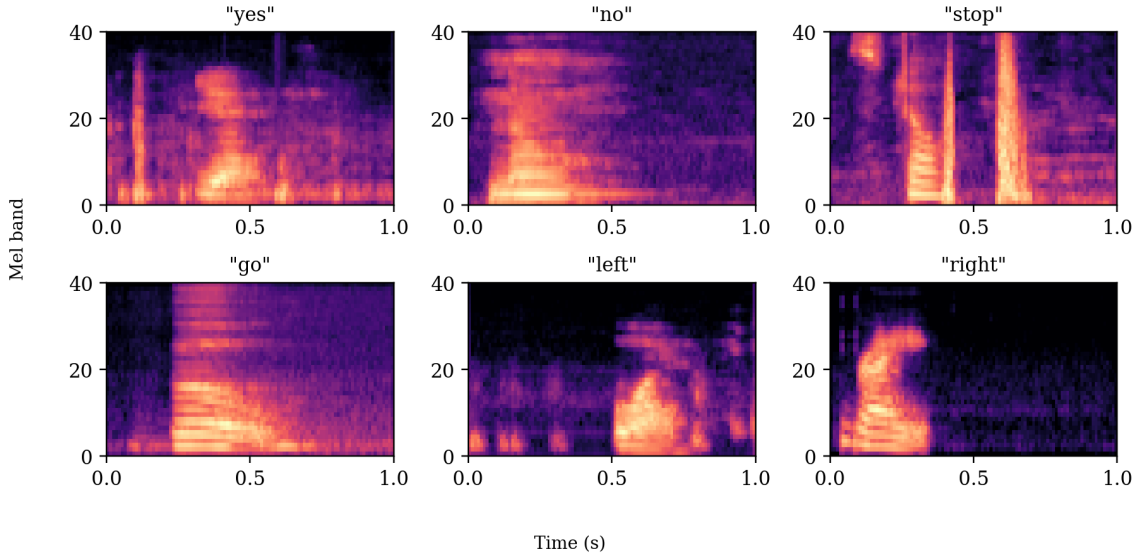


Figure 2.2: Log-mel spectrograms of six Google Speech Commands words (real data). Different words occupy visibly different time–frequency patterns, which is what makes metric learning in this space effective. *Generated from project data.*

Figure 2.3 makes the mel front-end concrete: (a) the 40 overlapping triangular filters that pool STFT energy — densely packed at low frequencies and sparse at high frequencies — and (b) the Hz-to-mel warping of Equation (2.1).

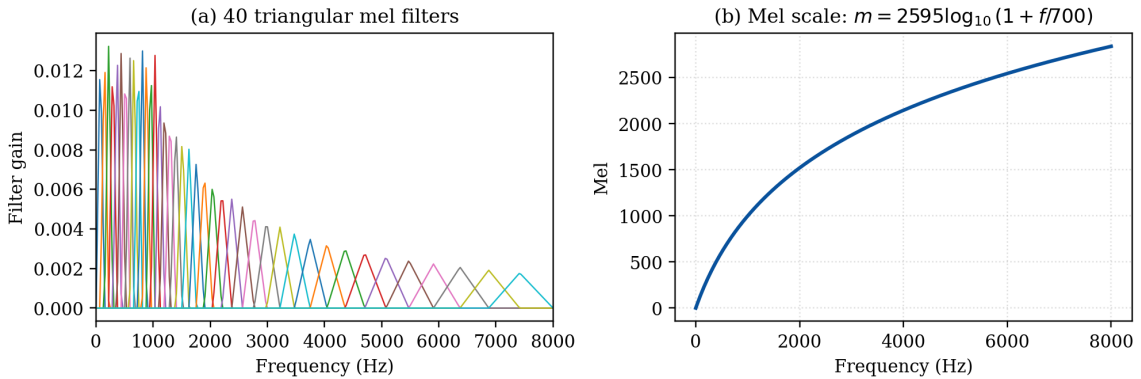


Figure 2.3: The mel front-end. (a) the 40-channel triangular mel filterbank used in this work; (b) the perceptual Hz→mel warping curve.

2.3 Keyword spotting: closed-set vs. open-set

Small-footprint KWS was popularized by depthwise-separable convolutional networks (DS-CNN) [1] and later refined by broadcasted-residual networks (BC-ResNet) [2], both of which trade accuracy against a small parameter count for microcontroller deployment. Fundamentally these are *closed-set* classifiers: the model outputs a probability distribution

over a *fixed* label set and always chooses the highest-probability label.

The core limitation is that a closed-set classifier has *no “none of the above” option*. The softmax function forces the probabilities over the known classes to sum to one, so every input — including a cough, music, or a word outside the list — is mapped to one of the known keywords. In laboratory conditions, where every test sample belongs to a known class, this is not exposed. In practice, however, the overwhelming majority of audio a device hears is *not* a keyword, so a closed-set classifier will continually mislabel background sound, causing false activations.

The *open-set* problem poses a harder and more realistic requirement: the system must both recognize the known keywords and *reject* any audio that does not belong to the keyword set. The challenge is that the space of “everything that is not a keyword” is essentially infinite and cannot be enumerated at training time, so it cannot be treated as an ordinary class. Instead, the system needs a *rejection mechanism*: it measures “how much an input resembles the known keywords” and rejects it when this measure is too low. The ability to quantify and tune the rejection threshold — accepting correct keywords while keeping the false-activation rate low — is exactly what separates a laboratory accuracy number from a deployable wake-word engine. This thesis pursues precisely that open-set scenario, with the concrete measurement defined in Section 3.5.

2.4 Few-shot learning and metric learning

Few-shot learning is the scenario in which a model must recognize a new class from only a very small number of examples (k shots). This is exactly the need of user-defined KWS: the user provides a few samples per keyword and expects the system to recognize that word immediately, without retraining. Since we cannot train a new classifier for every vocabulary, we need a representation that allows direct “matching” between a new sample and the few enrolled samples.

The natural approach is *metric learning*: instead of learning a mapping from audio to a fixed label, the network learns an *embedding space* in which distance reflects semantic similarity — utterances of the *same* word lie close together and utterances of *different* words lie far apart. Given such a space, a new keyword is represented simply by the positions of its enrollment samples, and recognition reduces to a nearest-point search — independent of whether the word appeared during training.

To make the model learn this property, training is organized into *episodes* (episodic training): each episode simulates a small few-shot problem by sampling a group of classes and splitting each class’s samples into a *support set* (playing the role of enrollment) and a *query set* (playing the role of the samples to be recognized). This mirrors the deployment condition, so the learned skill transfers well to real use. Prototypical networks [3] represent each class by the mean (*prototype*) of its support embeddings and then classify queries

by nearest prototype; triplet embeddings [4] instead optimize relative distances directly, pulling same-class pairs together and pushing different-class pairs apart. The Generalized End-to-End (GE2E) loss [5], originally proposed for speaker verification, builds centroids inside each training batch exactly as deployment builds prototypes; this match between the training objective and the deployment procedure makes GE2E particularly well suited to few-shot open-set KWS, which we confirm experimentally in Chapter 4.

2.5 Angular-margin classification and open-set rejection

Another family of methods for learning highly discriminative embeddings is *angular-margin classification*. The idea is to normalize both the embedding and the weight vector of each class to unit length, so that the classification score becomes the cosine of the angle between them; a *margin* is then added to the angle of the correct class to force the classes to separate more clearly on the hypersphere. ArcFace [6] applies this principle and the sub-center variant SCAF [7] lets each class have several sub-centers, improving robustness to noisy data; both are highly successful for face recognition with thousands of identities. The question that arises is whether this machinery transfers to tens of thousands of *word* classes — and this is the question the thesis answers *negatively* at extreme scale (Chapter 4): once the number of classes reaches tens of thousands, the angular-margin classification term dominates and collapses training unless it is strongly down-weighted.

Besides learning a good embedding, open-set KWS needs a *rejection mechanism* to decide when an input belongs to no keyword. The simplest is a threshold on the distance to the nearest prototype (*nearest-class-mean*): accept if close enough, reject if too far. More sophisticated methods are also studied: OpenMax [8] fits a Weibull tail distribution to the per-class distances to estimate a “belonging” probability, while the energy score [9] provides a calibration-free measure based on a weighted exponential sum of the distances. We adapt both to operate on prototype distances and compare them with the plain nearest-class-mean threshold (Section 3.4).

Finally, self-supervised speech models such as Wav2Vec 2.0 [11] are trained on large amounts of unlabeled data and provide very strong representations, but they are too large to run on an edge device. We therefore use a *frozen* Wav2Vec 2.0 only as a *teacher* in *knowledge distillation*: the teacher’s knowledge is transferred to a compact *student* encoder, keeping the deployed model very small while benefiting from the teacher’s representation (Section 3.3.3).

Chapter 3

Proposed Method

At a high level (Figure 3.1) the method has two phases that share a single, frozen encoder. In the *training phase* the encoder is optimized once on MSWC with a metric objective. In the *deployment phase* the user enrolls a few examples per keyword to form prototypes, and queries are recognized by nearest-prototype distance with an explicit open-set threshold — the encoder is never retrained.

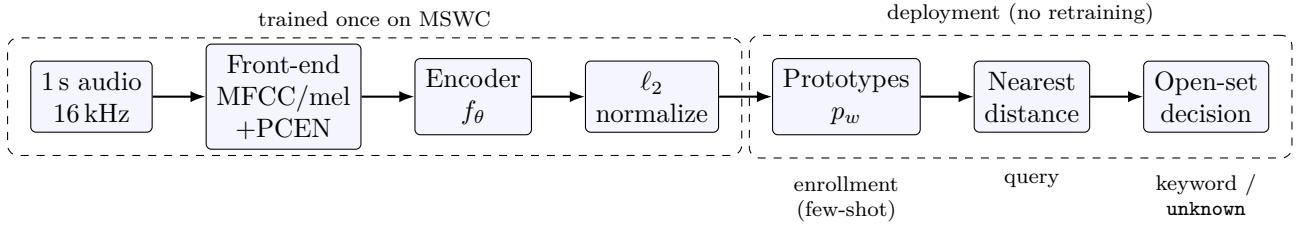


Figure 3.1: End-to-end few-shot open-set KWS pipeline. The encoder is trained once; at deployment, user keywords become prototypes and queries are classified by nearest-prototype distance with an open-set threshold.

3.1 Problem formulation

Let $f_\theta : \mathbb{R}^{T \times F} \rightarrow \mathbb{R}^d$ be an encoder mapping a one-second representation to a unit-norm embedding. The prototype of a keyword w is the mean of its k support embeddings,

$$p_w = \frac{1}{k} \sum_{i=1}^k f_\theta(x_i^{(w)}). \quad (3.1)$$

In Equation (3.1), $x_i^{(w)}$ is the i -th enrollment sample of keyword w , $f_\theta(\cdot)$ its embedding, and k the number of enrollment shots (here $k=10$). Intuitively, the prototype p_w is the central representative point of a keyword in embedding space; averaging several shots cancels the random variation of individual utterances and yields a stable description of the word — this is precisely the user’s “enrollment” operation.

A query x is assigned to the nearest prototype, with open-set score equal to the negated minimum distance,

$$\hat{w}(x) = \arg \min_w d(f_\theta(x), p_w), \quad s(x) = -\min_w d(f_\theta(x), p_w), \quad (3.2)$$

and accepted as $\hat{w}(x)$ iff $s(x) \geq \tau$, otherwise labelled **unknown**. In Equation (3.2), $d(\cdot, \cdot)$ is the distance in embedding space and τ the rejection threshold. The left expression determines *which* keyword is nearest; the right expression turns the distance to the nearest prototype into a *confidence score* $s(x)$ (close means high, far means low). The open-set decision is therefore intuitive: if the confidence exceeds τ , accept the nearest keyword; otherwise treat the input as “unknown”. Raising or lowering τ is exactly the knob that trades off missed keywords against false activations.

Figure 3.2 visualizes the learned space directly. Running the best trained encoder on real GSC recordings and projecting the embeddings to two dimensions, each known command forms a tight, well-separated cluster around its prototype (stars), while out-of-vocabulary words (grey crosses) fall in the gaps between clusters — exactly the geometry that makes nearest-prototype matching with distance thresholding work for open-set rejection.

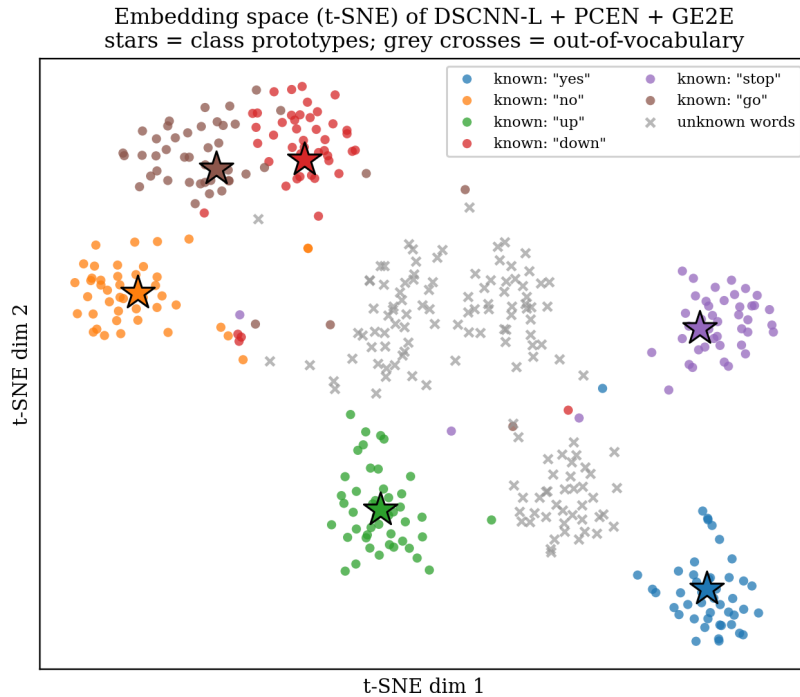


Figure 3.2: t-SNE projection of embeddings from the trained DSCNN-L + PCEN + GE2E model on real GSC audio. Coloured points are six known commands, stars are their prototypes, grey crosses are out-of-vocabulary words.

Worked example: enrollment, matching, and rejection. Suppose a user wants the device to recognize the command “stop”. *Enrollment*: the user records $k=10$ clips of

“stop”; each clip is passed through the encoder into an embedding, and their mean gives the prototype p_{stop} via Equation (3.1). Suppose the system has only this one keyword and the threshold is set so that the maximum accepted distance is 0.9 (i.e. $s(x) \geq \tau \Leftrightarrow d \leq 0.9$). *Matching*: when the user says “stop”, the query embedding lands close to p_{stop} , say $d = 0.45$; since $0.45 \leq 0.9$, the query is *accepted* and labelled “stop”. *Rejection*: when the microphone picks up an unfamiliar sound — for instance a piece of music or the out-of-vocabulary word “elephant” — its embedding lies far from every prototype, say $d = 1.6$; since $1.6 > 0.9$, the system *rejects* it and returns **unknown** instead of forcing a wrong “stop” label. This distance–threshold mechanism, applied simultaneously to many keywords, is the foundation of the open-set recognition in this thesis.

3.2 Datasets and Training Protocol

Training corpus (MSWC English): We train on the English subset of the Multilingual Spoken Words Corpus, which after filtering for $\geq$ minimum clips contains 37,387 training words (and 763 validation words), drawn from a metadata pool of $\sim 38,150$ words and ~ 6.6 M clips. To study scale we form caps of $\{20, 50, 220, 620\}$ clips per word — roughly $\{0.53, 0.94, 2.05, 2.99\}$ M training clips — and two reduced regimes: a Top-500 legacy split (450 train / 50 val words, evaluation on words ranked 501+ with ≥ 1000 utterances) and an MLCommons micro-set (31 keywords) used only for pipeline validation and architecture screening. The three vocabulary regimes span more than three orders of magnitude (Figure 3.3); this range is what makes the scale effects in Chapter 4 observable.

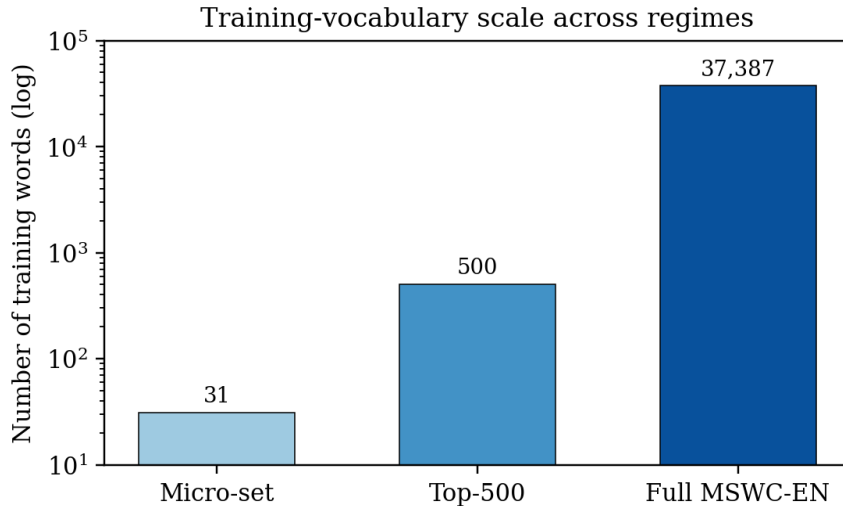


Figure 3.3: Training-vocabulary size for the three regimes (log scale). The micro-set is used only for screening; full MSWC English is the headline regime. *Generated from project data.*

The three regimes in Figure 3.3 serve three purposes. The *micro-set* (31 words) is small enough to train in minutes and is used to verify that the pipeline is correct and

to screen architecture choices quickly before committing to expensive runs. *Top-500* is an intermediate scale under the legacy split, which lets us compare against prior work that uses the same setup. *Full MSWC* (37,387 words) is the headline regime and reflects the realistic condition of a very large vocabulary; some phenomena — such as the SCAF collapse in Chapter 4 — appear only at this scale.

Episodic sampling and augmentation: Episodes draw 30 classes $\times$ 20 utterances (only classes with ≥ 20 clips are eligible). Online augmentation mixes DEMAND background noise (probability 0.5, SNR sampled in $[0, 10]$ dB) and applies SpecAugment (one frequency mask of width 6, one time mask of width 8); gain perturbation (± 6 dB) and time-shift (≤ 100 ms) are applied with probability 0.5. Figure 3.4 illustrates these augmentations on a real recording: additive background noise at 5 dB SNR fills the quiet regions, and the SpecAugment frequency/time masks (black bands) force the encoder not to rely on any single band or instant.

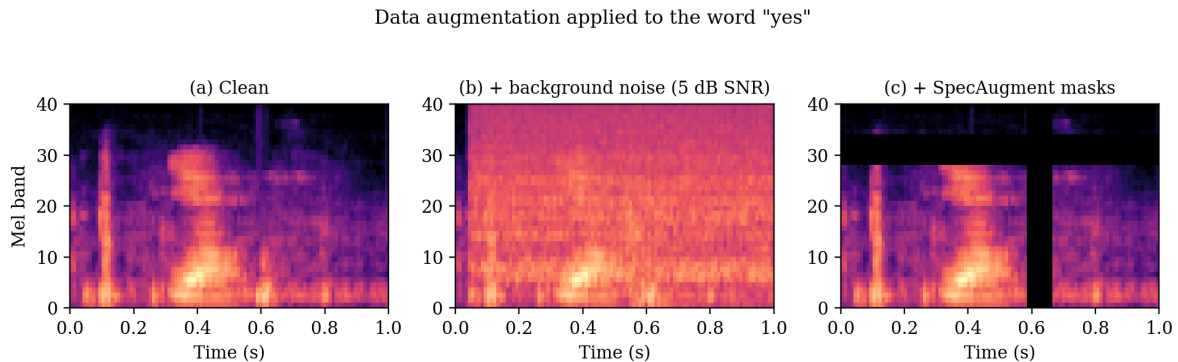


Figure 3.4: Training-time augmentation on the word “yes” (real data): (a) clean log-mel; (b) + real background noise at 5 dB SNR; (c) + SpecAugment frequency and time masks. *Generated from project data.*

Evaluation corpus (GSC v2): We evaluate cross-corpus on Google Speech Commands v2. Enrollment (support) samples are drawn from the official `validation_list.txt`. Query samples are drawn according to the evaluation split: in the “test” split they come from `testing_list.txt`, whereas in the “dev” split they come from the files *not* listed in either `validation_list.txt` or `testing_list.txt`. Because the three lists are disjoint, there is no leakage between enrollment and query samples. A synthetic `_silence_` class is generated from random one-second crops of `_background_noise_`, with disjoint crops for support and query.

EdgeSpot-exact protocol: The headline protocol partitions GSC’s 35 words into 11 positive classes — the ten command words {yes, no, up, down, left, right, on, off, stop, go} plus `_silence_` — and 25 negative (unknown) spoken words. Each run enrolls $k=10$ shots per positive word, computes prototypes, and scores up to 50 query samples per word (a fixed-seed subsample). We report the mean $\pm$ std over 100 randomized runs; randomness comes from enrollment-shot selection (the known/unknown vocabulary itself is fixed across

runs). “test100” denotes 100 runs on the test split, “dev30” denotes 30 runs on the dev split.

Table 3.1: The two corpora: MSWC English (training) and Google Speech Commands v2 (evaluation). The model input and output are identical for both corpora because they share the same feature front-end.

Property	MSWC English	Google Speech Commands v2
Role	Training the encoder	Cross-corpus evaluation
Source / citation	Mazumder <i>et al.</i> , 2021 [12]	Warden, 2018 [13]
Vocabulary	37,387 train / 763 val words (pool ~38,150)	35 words: 11 positive (10 commands + <code>_silence_</code>) + 25 negatives
Clips	~6.6 M (metadata); capped manifests {20, 50, 220, 620}/word → {0.53, 0.94, 2.05, 2.99} M	~105 k utterances
Sample rate	16 kHz, mono	16 kHz, mono
Clip length	1 s (padded/trimmed to 16,000 samples)	1 s (padded/trimmed to 16,000 samples)
Model input	16,000-sample waveform → features (1, 47, 10) [MFCC] or (1, 40, 101) [log-mel/PCEN]	
Model output	ℓ_2 -normalized embedding $d \in \{276, 64\}$ → prototype distance → keyword / unknown	
Usage	Caps {20, 50, 220, 620}/word; legacy Top-500; 31-word micro-set (screening)	EdgeSpot-exact 10-shot, up to 50 queries/word, 100 runs

Table 3.1 summarizes the two corpora used in this thesis. The key point is the *cross-corpus evaluation*: the model is trained entirely on MSWC English and tested on GSC v2 — a corpus independent in speakers, devices, and vocabulary — so the results reflect genuine generalization rather than fitting a single dataset.

3.3 System Architecture

This section describes the three components that make up the encoder and the decision procedure: Section 3.3.1 presents the feature front-ends that turn the waveform into a time–frequency representation; Section 3.3.2 describes the two encoder architectures (DSCNN-L and EdgeSpot-Full); and Section 3.3.3 presents the metric-learning objectives used to train them. The order follows the signal from input, through the network, to the embedding consumed by the deployment classifier.

3.3.1 Feature front-ends

All audio is mono, 16 kHz, and trimmed/zero-padded to exactly one second (16,000 samples). We support two spectral parameterizations, summarized in Table 3.2.

Table 3.2: Feature front-end configurations.

	MFCC	Log-mel
Sample rate	16 kHz, mono, 1 s	
FFT size	1024	512
Window / hop	40/20 ms	25/10 ms
center	False	True
Mel bands	40	40
Output coeffs.	10 (first DCT)	40 (mel)
Tensor shape	(1, 47, 10)	(1, 40, 101)
Trainable PCEN	optional	optional

MFCC (DSCNN front-end): We compute 40 Mel-frequency cepstral coefficients with a 1024-point FFT, 640-sample (40 ms) Hann window, 320-sample (20 ms) hop, and **center=False**; the Mel filterbank uses Slaney scale and Slaney area normalization, and the log is taken as $10 \log_{10}(\cdot)$ followed by an orthonormal DCT-II. We retain only the first 10 coefficients and transpose to a tensor of shape $(1, 47, 10) = (\text{channel}, T_{\text{frames}}, \text{features})$, where $T = 47 = \lfloor (16000 - 1024)/320 \rfloor + 1$.

Log-mel (EdgeSpot/BC-ResNet front-end): We use 40 Mel bands with a 512-point FFT, 400-sample (25 ms) window, 160-sample (10 ms) hop and **center=True**, producing $(1, 40, 101)$. The log compression is delegated to the PCEN layer.

Trainable PCEN: Per-channel energy normalization [10] applies a causal first-order IIR smoother $M_t = (1 - s)M_{t-1} + sE_t$ followed by adaptive gain and dynamic-range compression,

$$\text{PCEN}(E_t) = \left(\frac{E_t}{(\epsilon + M_t)^\alpha} + \delta \right)^r - \delta^r, \quad (3.3)$$

with learnable α, δ, r, s (initialized to 0.96, 2.0, 0.5, 0.04; $\epsilon=10^{-6}$). In Equation (3.3), E_t is the mel-band energy at time frame t and M_t its time-smoothed version (an estimate of the local background level). The roles of the components are as follows: the division $E_t/(\epsilon + M_t)^\alpha$ performs *automatic gain control* — normalizing each band by its recent background so that abrupt changes (consonant onsets) are emphasized and steady components (background noise) are suppressed; the exponent r together with the offset δ performs a soft *dynamic-range compression* in place of the static logarithm. Because α, δ, r, s are learned jointly with the network, PCEN adapts to the recording conditions instead of using a fixed compression function — which is why it is more robust and, as Chapter 4 shows, the single most reliable improvement in this study.

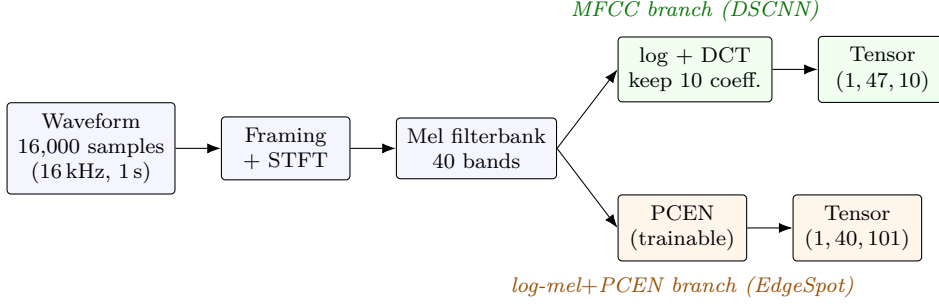


Figure 3.5: Preprocessing flow: 1 s waveform $\rightarrow$ framing/STFT $\rightarrow$ 40-band mel filterbank $\rightarrow$ two branches: log+DCT keeping 10 coefficients (MFCC, tensor (1, 47, 10)) or trainable PCEN (log-mel, tensor (1, 40, 101)).

Figure 3.5 summarizes the entire preprocessing pipeline as a flow diagram, from the one-second waveform to the feature tensor the encoder receives, with two branches corresponding to the two front-ends.

3.3.2 Encoders

Table 3.3: Encoder comparison. ℓ_2 -normalization is applied outside the encoder so all losses and the deployment classifier operate on the hypersphere.

Encoder	Front-end	d	Params
DSCNN-L	MFCC / mel(+PCEN)	276	0.413 M
EdgeSpot-Full $\tau=4$	mel(+PCEN)	64	0.131 M
EdgeSpot-Lite	mel(+PCEN)	64	< 0.131 M
BC-ResNet-FS $\tau=4$	mel(+PCEN)	64	~ 0.1 M

DSCNN-L: A depthwise-separable CNN built from a size table: an initial Conv (1 $\rightarrow$ 276, 10 $\times$ 4, stride 2 $\times$ 1) with TensorFlow-style asymmetric “same” padding, followed by five depthwise separable blocks (276 channels, 3 $\times$ 3 kernels; the first block strides 2 $\times$ 2). All but the last block use BatchNorm+ReLU; the last block uses an (affine-free) LayerNorm. A global average pool yields the embedding directly, so the embedding dimension equals the channel width, $d = 276$ (there is no final linear projection). DSCNN-L has ≈ 0.413 M parameters.

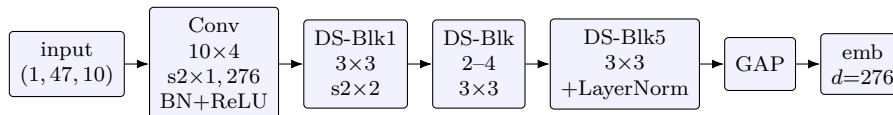


Figure 3.6: DSCNN-L encoder: depthwise-separable CNN, ≈ 0.41 M params. The embedding equals the final channel width (276); no linear projection.

EdgeSpot-Full ($\tau=4$): A width-scaled edge encoder operating on the (1, 40, 101) mel/PCEN input. A 5 $\times$ 5 stride-(2, 1) stem feeds four stages of fused-temporal and

broadcasted-residual “lite” blocks with increasing temporal dilation (1, 2, 4, 8); a depth-wise frequency-collapse block averages over frequency to a (C, T) sequence; a depthwise positional convolution and a single-head temporal self-attention block ($d = 64$) model long-range temporal structure; finally a depthwise temporal head and global pooling produce a 64-dimensional embedding. EdgeSpot-Full $\tau=4$ has ≈ 0.131 M parameters, $\sim 3.2\times$ smaller than DSCNN-L. A lighter EdgeSpot-Lite variant and a clean BC-ResNet-FS baseline (no attention) are also provided.

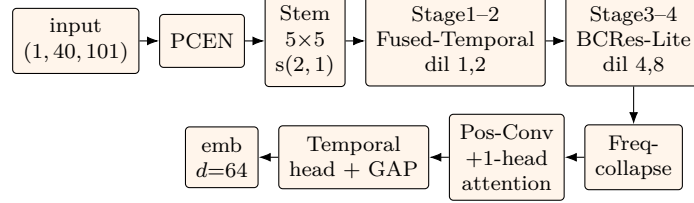


Figure 3.7: EdgeSpot-Full ($\tau=4$) encoder: mel/PCEN front-end, fused-temporal and broadcasted-residual blocks with growing dilation, frequency collapse, a single-head temporal self-attention stage, ≈ 0.13 M params.

3.3.3 Metric-learning objectives

All objectives are trained episodically: each episode samples $n_{\text{cls}}=30$ classes with $n_{\text{spp}}=20$ utterances each (600 embeddings/episode).

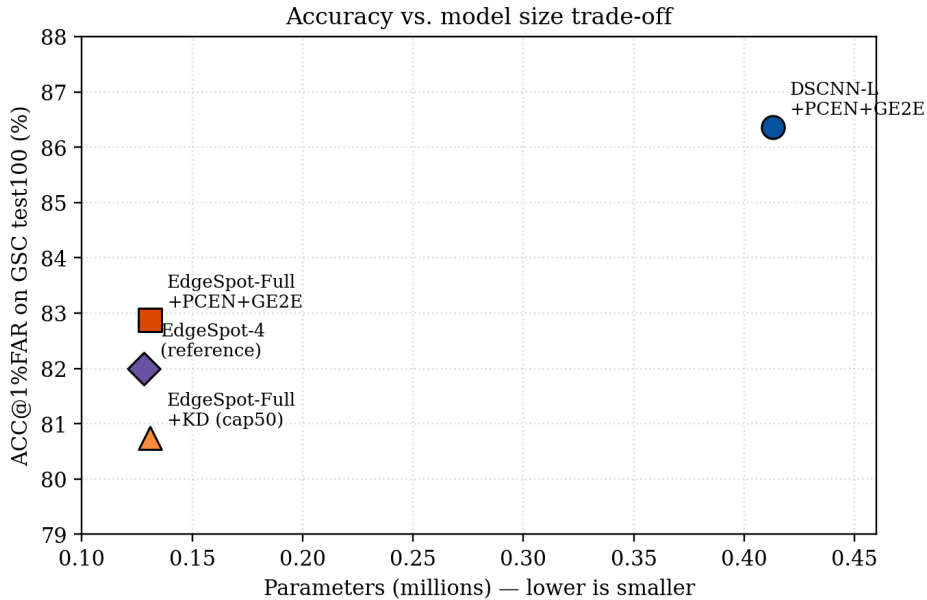


Figure 3.8: Accuracy vs. model size. DSCNN-L leads on accuracy; EdgeSpot-Full reaches a competitive point at one third the parameters; KD lifts the compact model further at low data. (Accuracy numbers are detailed in Chapter 4.) *Generated from project results.*

Triplet: Semi-hard triplet loss [4] with Euclidean distance, $\mathcal{L} = \frac{1}{N} \sum \max(0, d(a, p) - d(a, n) + m)$, with hardest-positive mining and FaceNet semi-hard negative mining. Here

a is the anchor, p a sample of the *same* word (positive), n a sample of a *different* word (negative), and m the margin. The loss forces the anchor–positive distance to be smaller than the anchor–negative distance by at least m ; the $\max(0, \cdot)$ means a triplet is penalized only when this condition is violated, i.e. the model learns only from “hard” triplets.

Sub-center ArcFace (SCAF): An angular-margin softmax with $K=3$ sub-centers per class. With normalized weights $W \in \mathbb{R}^{(C \cdot K) \times d}$, the per-class logit is the max cosine over its sub-centers, $\cos \theta_c = \max_{j \leq K} w_{c,j}^\top z$; the additive margin $m=0.5$ is applied to the target class and logits are scaled by $s=30$ before cross-entropy. Here z is the normalized embedding and $w_{c,j}$ the j -th sub-center of class c ; taking the max over sub-centers lets one word have several distinct “pronunciation styles”. The margin m forces the correct class to be clearly superior in angle, while the scale s amplifies the logits before softmax — and it is exactly these two factors that, multiplied across tens of thousands of classes, cause the collapse analyzed in Chapter 4.

GE2E (centroid): Generalized end-to-end loss [5] that mirrors deployment: within an episode each class is split into support/query; the centroid is the ℓ_2 -normalized support mean and queries are classified by scaled cosine similarity to all centroids with learnable scale and bias, optimized by cross-entropy. GE2E directly optimizes the enroll-then-match objective of Equations (3.1) and (3.2).

Cosine knowledge distillation (KD): We additionally distill from a frozen Wav2Vec 2.0-base teacher (hidden layer 16, mean-pooled, projected to $d=64$ by a sub-center-ArcFace-trained head). The distillation loss is a pure embedding-cosine term,

$$\mathcal{L}_{\text{KD}} = 1 - \frac{1}{N} \sum_i \cos(f_\theta(x_i), g_{\text{teacher}}(x_i)), \quad (3.4)$$

i.e. no temperature/soft-label term. In Equation (3.4), $f_\theta(x_i)$ is the student embedding and $g_{\text{teacher}}(x_i)$ the frozen Wav2Vec 2.0 teacher embedding for the same input x_i . Since $\cos$ equals 1 when two vectors point in the same direction, minimizing $\mathcal{L}_{\text{KD}}$ is equivalent to aligning the student embedding with the teacher embedding — i.e. forcing the small model to imitate the representation space of the large one. Teacher embeddings are precomputed offline and looked up by file path during training, so the additional training cost is negligible.

Hybrid combinations: We form weighted sums, e.g. `scaf_ge2e`, `kd_ge2e`, `kd_scaf`, `kd_scaf_ge2e`. A crucial implementation detail (and a key empirical finding, Chapter 4) is the SCAF weight: under KD it is set to 5×10^{-5} while GE2E/KD weights are 1.0. The very small SCAF weight prevents the high-magnitude angular-margin classification loss from dominating the centroid/cosine terms.

Optimization. Unless noted otherwise, every model is trained with the Adam optimizer (initial learning rate 10^{-3} , weight decay 10^{-4}). The gradient norm is clipped at 5.0 (gradient clipping) to avoid overly large update steps that would destabilize training,

especially for the hybrid losses. The learning rate decays on a *cosine-annealing schedule with warm restarts* (CosineAnnealingWarmRestarts): it follows a cosine curve downward and is then reset periodically, with a first cycle of $T_0=10$ epochs, each subsequent cycle $T_{\text{mult}}=2\times$ longer than the previous one, and a floor of $\eta_{\text{min}}=10^{-5}$. For reproducibility we fix all random seeds ($= 42$) and enable cuDNN deterministic mode. Per-experiment epoch/episode budgets are stated in Chapter 4.

3.4 Open-set rejection heads

All heads share the prototype interface and differ only in the rejection score; the predicted keyword is always the argmin-distance class (Equation (3.2)). (i) **Nearest-class-mean (NCM, “Direct ℓ_2 ”)** — our default — rejects when the minimum prototype distance exceeds a threshold, using raw distances with no probability normalization. (ii) **Open-Max** [8] fits a Weibull tail to support-to-prototype distances and uses $1 - \text{CDF}$ as the membership score. (iii) **Energy** [9] treats $-d_i$ as logits and uses the free energy $E(x) = T \log \sum_i \exp(-d_i/T)$ as the acceptance score. (iv) **Calibrated** thresholds ($\mu + 2\sigma$ of support distances) drive per-word confusion analysis.

3.5 Evaluation methodology

This section defines the evaluation metrics *before* they are used in Chapter 4. We treat the open-set problem as a binary keyword-vs-non-keyword detection augmented by a multiclass identity track, and build all metrics on the ROC curve. Two base concepts first: FAR (False-Accept Rate) is the fraction of *non-keyword* audio that is wrongly accepted, and FRR (False-Reject Rate) is the fraction of *genuine* keywords that are wrongly rejected. The symbol $\uparrow$ means *higher is better* and $\downarrow$ means *lower is better*.

- **AUC** ($\uparrow$): area under the ROC of the open-set scores (Equation (3.2)); threshold-free, it measures the general separability of keywords from non-keywords.
- **EER** ($\downarrow$): equal-error rate, the operating point minimizing $|\text{FAR} - \text{FRR}|$, reported as $(\text{FAR} + \text{FRR})/2$.
- **FRR@FAR** ($\downarrow$): false-reject rate at the largest threshold whose false-accept rate does not exceed the target (1% or 5%).
- **Open-set ACC@FAR** ($\uparrow$): the headline accuracy. A known query is correct iff it is accepted ($s \geq \tau_{\text{FAR}}$) and the argmin label is correct; an unknown query is correct iff it is rejected. Reported at $\text{FAR} \in \{1\%, 5\%\}$.
- **Keyword ACC** ($\uparrow$): closed-set nearest-prototype accuracy over known queries only (ignores rejection).

- **F1** ($\uparrow$): binary keyword-vs-unknown F1 at the EER threshold.

Interpretation caveat. AUC and EER are threshold-free, whereas ACC@FAR and FRR@FAR use a threshold derived from the evaluation ROC (an oracle operating point), not transferred from a held-out calibration split. We therefore treat ACC@FAR as a comparison metric across systems under matched conditions, and report AUC/EER/F1 jointly so that degenerate operating points are visible. A system that rejects nearly everything can score a high open-set ACC against many negatives while missing all positives; such cases are exposed by $\text{FRR} \rightarrow 100\%$ and $\text{F1} \rightarrow 0$.

3.6 Experimental design: what we measure

The experiments are organized around four questions:

- Q1 — Which design axis matters?** We train all $2 \times 2 \times 4 = 16$ combinations of $\{\text{DSCNN-L, EdgeSpot-Full}\} \times \{\text{MFCC, PCEN}\} \times \{\text{Triplet, SCAF, GE2E, SCAF+GE2E}\}$ on full-vocabulary MSWC and isolate each axis with *paired deltas* (changing one factor while holding the others fixed).
- Q2 — How does performance scale?** Fixing the best front-end and loss, we sweep the per-word cap $\{20, 50, 220, 620\}$ and grow the vocabulary from 31 to 37,387 words, watching for both saturation and failure.
- Q3 — Can distillation replace data?** We compare GE2E against the cosine-KD objective at matched data caps.
- Q4 — What is the best deployable system?** We consolidate the strongest configurations per data regime and identify the best overall and best compact operating points.

3.7 Deployment: streaming and demonstrator

Beyond the core encoder, the system includes a CPU streaming pipeline and an offline web demonstrator (Figure 3.9). Streaming performs sliding-window detection (1 s window, 0.5 s stride) gated by Silero Voice Activity Detection (VAD): only windows that VAD marks as speech are embedded and matched, reducing computation and false activations on silence. A backend-agnostic decision *state machine* (Algorithm 1) stabilizes raw candidates into clean events using majority-vote smoothing over a short window, a top-2 margin check, and a cooldown to suppress duplicate firings. Optional modules — a spectral-gating/SpeechBrain denoiser and an ECAPA-TDNN speaker-verification gate — follow the same fail-open, runtime-toggle pattern. The pipeline is designed to a per-component CPU budget (VAD < 5 ms/chunk, MFCC < 5 ms, encoder < 10 ms, classifier < 1 ms; core

< 21 ms; < 71 ms with all enhancements); a measured on-device benchmark is left to future work.

Algorithm 1 Streaming decision state machine (smoothing + cooldown).

Require: window W , votes V , margin $m_{\min}$, cooldown C

```

1: buffer  $\leftarrow$  deque(maxlen =  $W$ );  $t_{\text{last}} \leftarrow -\infty$ 
2: for each candidate  $c$  with span  $[t_s, t_e]$  do
3:   if  $t_s - t_{\text{last}} < C$  then
4:     emit COOLDOWN; continue
5:   end if
6:   if  $\neg c.\text{detected}$  or  $c.\text{kw} = \text{unknown}$  or  $c.\text{margin} < m_{\min}$  then
7:     clear buffer; emit REJECTED; continue
8:   end if
9:   push  $c$ ;  $k^* \leftarrow$  majority keyword in buffer
10:  if  $\text{votes}(k^*) \geq V$  then
11:     $a \leftarrow \arg \max_{\text{conf}} \{c \in \text{buffer} : c.\text{kw} = k^*\}$ 
12:     $t_{\text{last}} \leftarrow a.t_e$ ; clear buffer; emit DETECTED( $k^*$ )
13:  else
14:    emit SCORING
15:  end if
16: end for

```

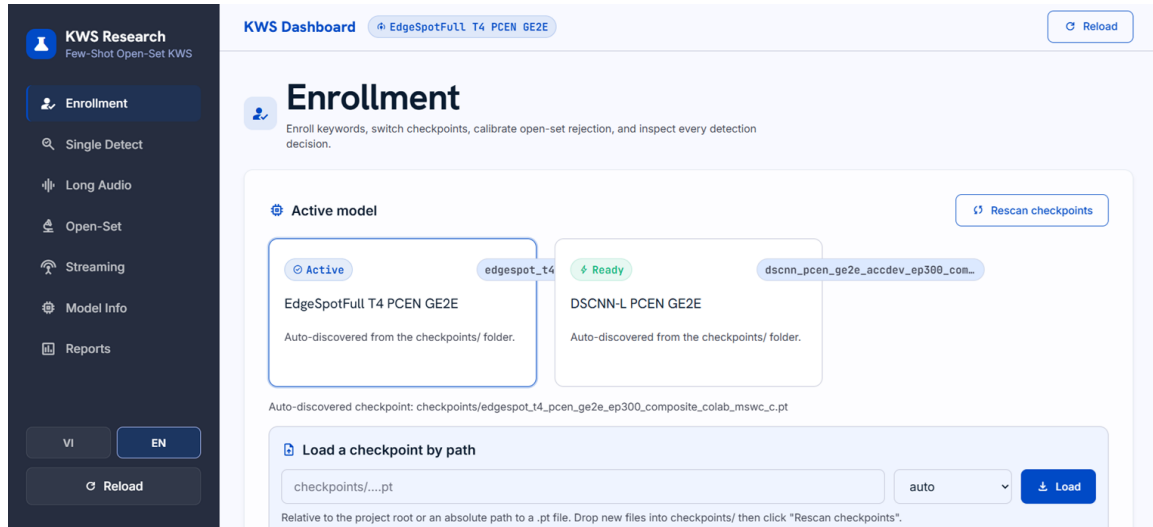


Figure 3.9: The offline web demonstrator: a model switcher and a long-audio timing view in which detected keywords are marked along the recording. *Screenshot of the project’s demo.*

Chapter 4

Results and Analysis

This chapter presents and analyzes the experimental results; the metrics were defined in Section 3.5, so the discussion here is purely numerical. Section 4.1 first inspects the training dynamics to confirm a healthy optimization. The results are then organized by the four research questions from Section 3.6: Section 4.2 (Q1) isolates which design axis matters; Section 4.3 (Q2) analyzes scale effects, including data saturation and the SCAF collapse; Section 4.4 (Q3) asks whether distillation can substitute for part of the data; and Section 4.5 (Q4) consolidates the best deployable systems. Finally, Section 4.6 validates the system end-to-end on a long recording.

Reporting conventions. Unless stated otherwise, all numbers are mean $\pm$ standard deviation (std) over the EdgeSpot-exact 10-shot protocol with 100 runs on GSC test (denoted *test100*); the std reflects stability across enrollment-shot selections. The best number in each column is shown in **bold**. As defined in Section 3.5, higher is better for ACC/AUC/F1/Keyword-ACC and lower is better for EER/FRR. Rows marked “coll.” are configurations that suffered a degenerate collapse ($\text{AUC}\approx 50$, $\text{F1}\approx 0$, $\text{FRR}\approx 100\%$) and are *not* treated as good results despite an open-set ACC near 69% (which is reject-all behaviour).

4.1 Training dynamics

Figure 4.1 shows the real training trajectory of the Microset anchor model (EdgeSpot-Full + SCAF+GE2E), read directly from the saved TensorBoard logs. The SCAF+GE2E loss decreases smoothly from ~ 15.5 to ~ 1.3 , the validation AUC rises to and stabilizes near 98%, and the GSC-dev open-set accuracy plateaus around 85% — a healthy, non-overfitting trajectory. (The contrasting *collapse* behaviour at full 37k-class vocabulary is discussed in Section 4.3.)

Training dynamics of the Microset anchor (EdgeSpot-Full + SCAF+GE2E)

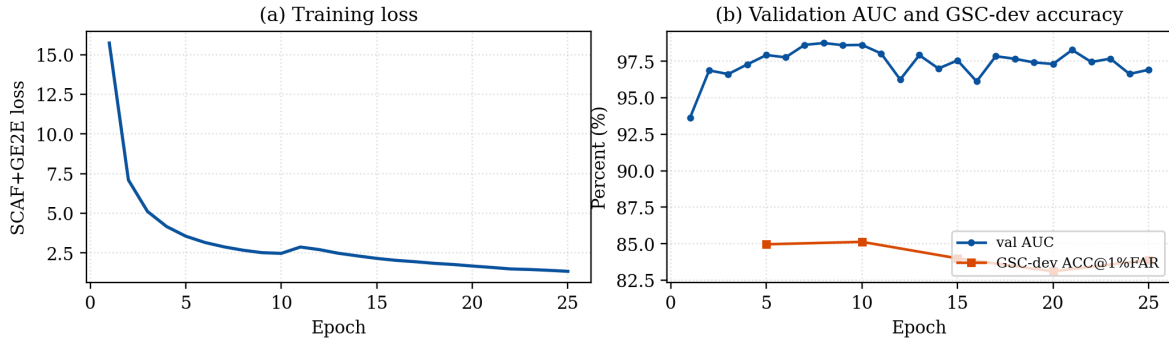


Figure 4.1: Real training dynamics of the Microset anchor (EdgeSpot-Full + SCAF+GE2E), from local TensorBoard logs: (a) training loss; (b) validation AUC and GSC-dev ACC@1%FAR over epochs.

4.2 Q1: Which design axis matters

Table 4.1: Full 16-configuration screen on full-vocabulary MSWC (cap-620), GSC test100 @ 1% FAR. Values are mean $\pm$ std over 100 runs. “coll.” marks degenerate collapse (AUC=50, F1=0, FRR=100%); std= 0 on these rows reflects deterministic reject-all behaviour. Best in bold.

Enc.	Feat.	Loss	ACC _{1%}	AUC	EER	F1
DSCNN	MFCC	Triplet	71.52 $\pm$ 0.67	75.55 $\pm$ 1.09	31.41 $\pm$ 0.98	57.17 $\pm$ 1.11
DSCNN	MFCC	SCAF	70.08 $\pm$ 0.44	55.04 $\pm$ 1.11	46.41 $\pm$ 0.85	41.38 $\pm$ 0.83
DSCNN	MFCC	GE2E	77.08 $\pm$ 1.24	86.46 $\pm$ 0.83	21.95 $\pm$ 1.03	68.50 $\pm$ 1.30
DSCNN	MFCC	SCAF+GE2E	69.04 $\pm$ 0.41	47.78 $\pm$ 1.21	52.15 $\pm$ 1.67	35.93 $\pm$ 1.54
DSCNN	PCEN	Triplet	79.98 $\pm$ 1.11	90.65 $\pm$ 0.62	17.57 $\pm$ 0.93	74.14 $\pm$ 1.23
DSCNN	PCEN	SCAF	69.44 $\pm$ 0.00	50.00 $\pm$ 0.00	50.00 $\pm$ 0.00	coll.
DSCNN	PCEN	GE2E	82.34$\pm$1.19	92.42$\pm$0.54	14.89$\pm$0.84	77.75$\pm$1.15
DSCNN	PCEN	SCAF+GE2E	69.44 $\pm$ 0.00	50.00 $\pm$ 0.00	50.00 $\pm$ 0.00	coll.
Edge	MFCC	Triplet	69.63 $\pm$ 0.34	52.84 $\pm$ 0.95	48.05 $\pm$ 1.02	39.79 $\pm$ 0.98
Edge	MFCC	SCAF	69.44 $\pm$ 0.00	50.00 $\pm$ 0.00	50.00 $\pm$ 0.00	coll.
Edge	MFCC	GE2E	70.76 $\pm$ 0.39	65.30 $\pm$ 1.12	39.05 $\pm$ 1.07	48.82 $\pm$ 1.12
Edge	MFCC	SCAF+GE2E	69.67 $\pm$ 1.01	50.88 $\pm$ 0.90	50.39 $\pm$ 0.82	37.57 $\pm$ 0.76
Edge	PCEN	Triplet	79.58 $\pm$ 1.35	89.85 $\pm$ 0.63	18.22 $\pm$ 0.78	73.29 $\pm$ 1.02
Edge	PCEN	SCAF	69.44 $\pm$ 0.00	50.00 $\pm$ 0.00	50.00 $\pm$ 0.00	coll.
Edge	PCEN	GE2E	79.98$\pm$0.98	87.23$\pm$0.75	20.23$\pm$0.96	70.68$\pm$1.23
Edge	PCEN	SCAF+GE2E	69.44 $\pm$ 0.00	50.00 $\pm$ 0.00	50.00 $\pm$ 0.00	coll.

Table 4.1 reports the full 16-configuration screen on full-vocabulary MSWC (cap-620), and Figure 4.2 ranks the configurations visually. Two findings are immediate: PCEN-based pipelines occupy the top of the ranking, and the centroid GE2E loss is the best objective for every backbone. The classification objective SCAF (and the SCAF+GE2E hybrid at full weight) *collapses* at this scale — a phenomenon analyzed in Section 4.3.

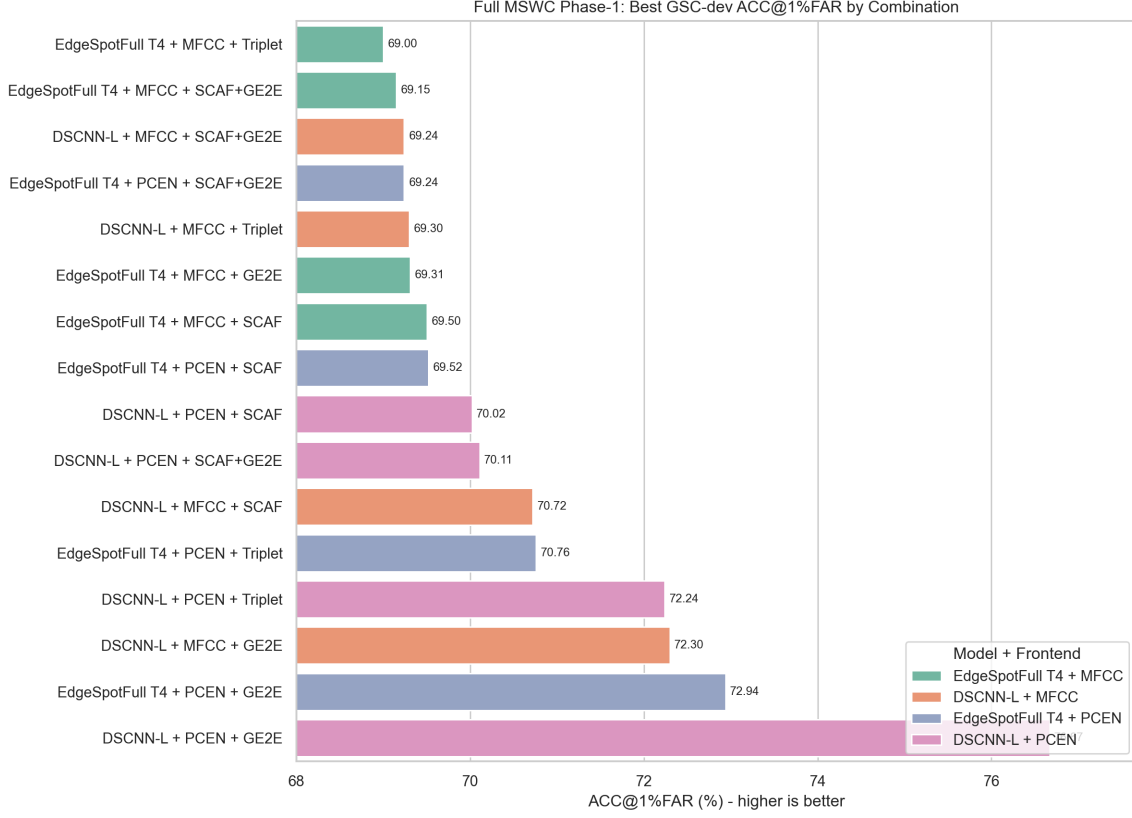


Figure 4.2: Configurations ranked by GSC ACC@1%FAR (phase-1 screen on full MSWC). PCEN+GE2E pipelines lead; colour encodes backbone×front-end. *Project-generated result figure.*

How to read the screening table. Table 4.1 is a *screening* run with a fixed training budget that is identical across all 16 configurations, so its purpose is the *relative* comparison of the design axes rather than absolute peak performance. The absolute values here are therefore systematically lower than the longer, specialized runs (ep300, checkpoint selected by a composite metric) consolidated in Table 4.5: for example, DSCNN-L+PCEN+GE2E reaches 82.34 in the screening table but 86.36 (+4.02 points) when trained fully. The *ranking* of the axes is consistent between the two regimes.

Paired effect deltas. Isolating each axis (Table 4.2, Figure 4.3) shows two robust wins. PCEN improves every backbone — most dramatically the edge encoder (+9.22 ACC, +21.86 F1) — and GE2E consistently beats triplet. The front-end therefore carries proportionally more of the representational burden when the backbone is small.

Table 4.2: Paired effect deltas (percentage points), full-vocabulary MSWC, GSC test100 @ 1% FAR. Positive favours the first option.

Contrast (others fixed)	$\Delta\text{ACC}_{1\%}$	ΔAUC	ΔF1
PCEN – MFCC (DSCNN, GE2E)	+5.26	+5.96	+9.25
PCEN – MFCC (Edge, GE2E)	+9.22	+21.93	+21.86
GE2E – Triplet (DSCNN, PCEN)	+2.36	+1.77	+3.61
DSCNN – Edge (PCEN, GE2E)	+2.36	+5.19	+7.07

Practical implication. The two “cheap” choices — switching the front-end to PCEN and the objective to GE2E — deliver most of the benefit without increasing model size. For a product team this means a device using a small edge encoder can substantially raise its reliability simply by changing the preprocessing block and the training objective, rather than moving to a larger, more power-hungry model.

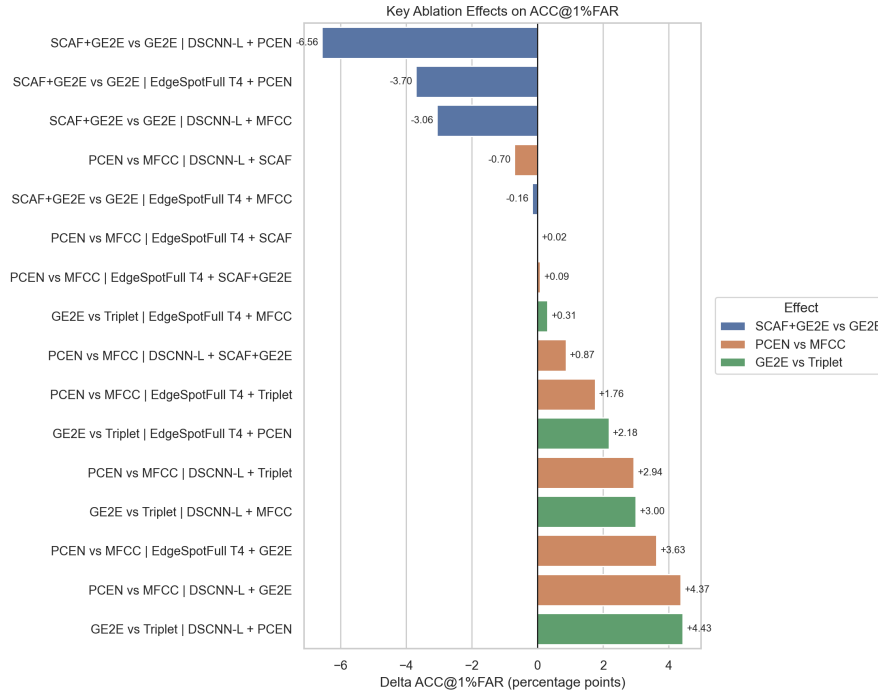


Figure 4.3: Key paired effect deltas. PCEN and GE2E are the two consistent improvements across backbones. *Project-generated result figure.*

4.3 Q2: Scale — saturation and the SCAF collapse

Table 4.3: Data-scale sweep (PCEN+GE2E), GSC test100, ACC at 5% FAR. Each cap is a separate training run (budgets differ across caps); the cap- $\{20, 50, 220\}$ rows include std over 100 evaluation runs, while the cap-620 row is taken from a saturation run for which per-run std was not recorded.

Encoder	Cap	ACC _{5%}	AUC	EER	F1
DSCNN-L	20	86.05±0.66	91.57±0.58	16.25±0.86	75.90±1.16
	50	84.68±0.70	90.45±0.68	17.42±1.08	74.34±1.43
	220	88.23±0.68	93.87±0.47	12.78±0.80	80.67±1.12
	620	88.56	94.04	12.53	81.02
EdgeSpot	20	83.06±0.82	87.22±0.75	20.40±1.01	70.46±1.30
	50	82.24±0.74	87.74±0.66	20.19±0.90	70.73±1.16
	220	86.03 ±0.70	91.31±0.62	16.47±0.78	75.61±1.06
	620	86.01	91.34	16.64	75.38

Accuracy saturates near 2 M clips. Figure 4.4 and Table 4.3 sweep the per-word cap for the best front-end/loss (PCEN+GE2E). There is a large jump between cap-50 and cap-220 (+3.55 ACC for DSCNN, +3.79 for EdgeSpot at 5% FAR) and a flat region from cap-220 to cap-620 (+0.33/−0.02). Accuracy is effectively data-saturated at ~ 2 M training clips: beyond this point, more clips of the same words yield negligible cross-corpus gains.

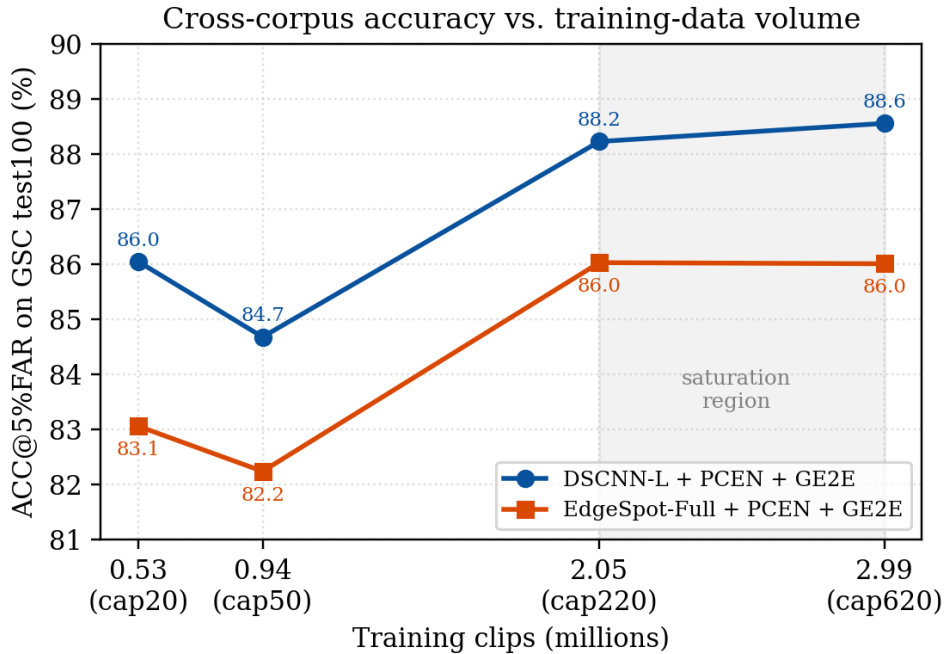


Figure 4.4: Cross-corpus accuracy vs. training-data volume (per-word caps). Performance saturates near ~ 2 M clips (shaded). *Generated from project results.*

The SCAF collapse at 37k classes. The most striking result of the scale study is a qualitative failure. On the 31-class micro-set and the ~ 500 -class Top-500 split, SCAF and SCAF+GE2E are excellent (AUC ≈ 95 ; Section 4.5). However, on the full 37,387-class vocabulary with the default SCAF weight of 1.0, training collapses from epoch 2: validation AUC pins at 0.5000 and the in-episode GE2E accuracy falls to ≈ 0.033 ($\approx 1/30$, i.e. chance). The cross-corpus GSC metrics are correspondingly degenerate (Figure 4.5): AUC 50, F1 0, FRR=100%, keyword ACC 9.09% ($= 1/11$ classes), while the open-set ACC@1%FAR stays at a *misleading* 69.4% — a reject-all artefact that AUC, F1 and FRR immediately expose. The mechanism is a gradient-magnitude imbalance: the sub-center ArcFace logit scale ($s=30$) multiplied across $\sim 3.7 \times 10^4$ classes produces a classification gradient that dwarfs the centroid/cosine terms, driving all embeddings to a trivial solution. The same hybrid does *not* collapse under KD, where the SCAF weight is 5×10^{-5} . The practical remedies we verify are: (i) reduce the SCAF weight to $\sim 5 \times 10^{-5}$, (ii) warm up the margin/scale, or (iii) drop the classification term and use GE2E or KD. This is a cautionary result: angular-margin classification losses that are state-of-the-art at 10^2 – 10^3 classes do not transfer naively to 10^4 -class word embedding.

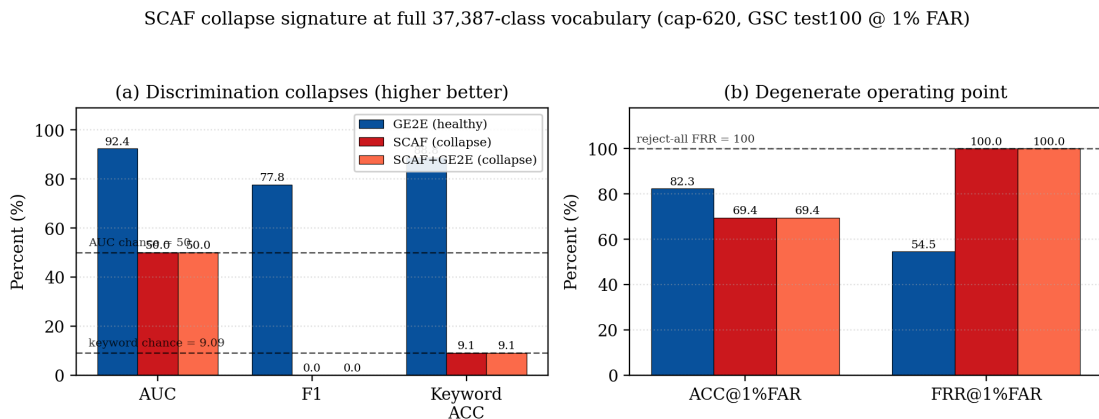


Figure 4.5: Collapse signature of SCAF and SCAF+GE2E at the full 37,387-class vocabulary (cap-620, GSC test100 @ 1% FAR), contrasted with the healthy DSCNN-L+PCEN+GE2E configuration. (a) Discrimination metrics floor out: AUC drops to chance (50), F1 to 0 and keyword ACC to 9.09% ($= 1/11$). (b) The operating point is degenerate: the collapsed runs reject everything (FRR=100%), yet open-set ACC@1%FAR stays a misleading $\sim 69\%$ against the many negatives. All values are the real cap-620 final metrics (same source as Table 4.1).

Practical implication. Both Q2 findings have clear operational consequences. Saturation near ~ 2 M clips means that, beyond a certain point, collecting more clips per word barely improves accuracy — so a data-collection budget should prioritize *diversity of words and recording conditions* over more clips of the same word. The SCAF collapse at large scale is a cautionary tale: a loss that is excellent at small scale can destroy training at real scale unless re-tuned, so design choices must be re-validated whenever the scale increases rather than assumed to still hold.

4.4 Q3: Knowledge distillation trades teacher knowledge for data

Table 4.4: Knowledge distillation vs. GE2E (EdgeSpot student), GSC test100. “n/a” means ACC@1%FAR was not recorded for this cap-220 GE2E run (only ACC@5%FAR); the cap-220 comparison therefore relies on the ACC@5%FAR column.

Cap	Loss	ACC _{1%}	ACC _{5%}	EER	F1
50	GE2E	77.14	82.24	20.19	70.73
50	KD	80.74	85.82	15.41	77.04
220	GE2E	n/a	86.03	16.47	75.61
220	KD	80.82	85.90	15.36	77.10

Table 4.4 distills a frozen Wav2Vec 2.0 teacher into the compact EdgeSpot student. At cap-50, KD improves every metric over GE2E (+3.60 ACC@1%FAR, −4.78 EER, +6.31 F1) and, strikingly, KD at cap-50 matches GE2E at cap-220 — the same accuracy with roughly half the training clips. KD also saturates early (cap-50→cap-220 is essentially flat), indicating that the teacher injects most of its useful inductive bias from limited data.

Practical implication. This result matters when data are scarce: a strong teacher model can compensate for missing data, achieving the same accuracy with roughly half the clips. For a development team without a large per-word dataset, distillation is a practical way to cut data-collection cost while keeping the deployed model compact.

4.5 Q4: Best deployable systems

Table 4.5: Consolidated best systems across data regimes (GSC test100, EdgeSpot-exact 10-shot, 100 runs). “coll.”= degenerate collapse; “n/a”= ACC@1%FAR not recorded for that run. For readability, the std ($\pm$) of the three headline configurations is given in the text above (roughly ± 0.4 to ± 1.3 points); the remaining rows are a single training run per regime.

Regime	Encoder	Loss	Params	ACC _{1%}	ACC _{5%}	AUC	EER	F1	KW
Micro (31)	EdgeSpot	SCAF+GE2E	0.131M	84.61	86.12	95.61	11.54	82.41	77.66
Top-500	EdgeSpot	SCAF+GE2E ep13	0.131M	85.62	88.79	95.34	11.51	82.45	90.44
Top-500	DSCNN-L	GE2E	0.413M	81.55	86.56	93.17	14.00	78.97	88.62
Full cap20	DSCNN-L	GE2E	0.413M	82.10	86.05	91.57	16.25	75.90	88.90
Full cap220	DSCNN-L	GE2E	0.413M	n/a	88.23	93.87	12.78	80.67	89.82
Full cap620	DSCNN-L	GE2E ep300	0.413M	86.36	89.93	95.21	11.32	82.73	92.92
Full cap620	EdgeSpot	GE2E ep300	0.131M	82.87	86.76	92.41	14.82	77.85	87.29
Full cap50	EdgeSpot	KD	0.131M	80.74	85.82	91.19	15.41	77.04	87.95
Full cap220	EdgeSpot	SCAF+GE2E	0.131M	coll.	coll.	50.00	50.00	coll.	9.09

Table 4.5 consolidates the strongest configurations per data regime, and Figures 4.6a and 4.6b visualize the headline operating point. Unlike the fixed-budget screening table

in Section 4.2, the cap-620 “ep300” rows here are specialized runs that are trained fully and have their checkpoint selected by a composite GSC-dev metric, so they reach a higher absolute peak. Three points stand out:

- **Best overall:** DSCNN-L+PCEN+GE2E (cap-620) reaches 86.36 ± 1.29 ACC@1%FAR and 89.93 ± 0.65 at 5% FAR, with AUC 95.21, EER 11.32, F1 82.73, keyword ACC 92.92%.
- **Best compact:** EdgeSpot-Full+PCEN+GE2E (cap-620) reaches 82.87 ± 1.22 ACC@1%FAR at 0.131 M parameters — within ~ 3.5 points of the best system at one third the size, and on par with the EdgeSpot-4 literature reference ($\sim 82\%$ ACC@1%FAR, $\sim 128k$ params).
- **Best small-vocabulary:** on the reduced Top-500 split, the EdgeSpot+PCEN+SCAF+GE2E hybrid is the strongest compact model (85.62 ± 1.04 ACC@1%FAR, AUC 95.34), confirming that the hybrid is excellent *when the class count is small enough not to trigger the collapse*.

Figure 4.6 summarizes the chapter’s story from two complementary angles: the DET curve shows that the best system keeps a low miss rate across a wide range of false-alarm rates, while the heat map confirms that PCEN+GE2E is the brightest region regardless of backbone.

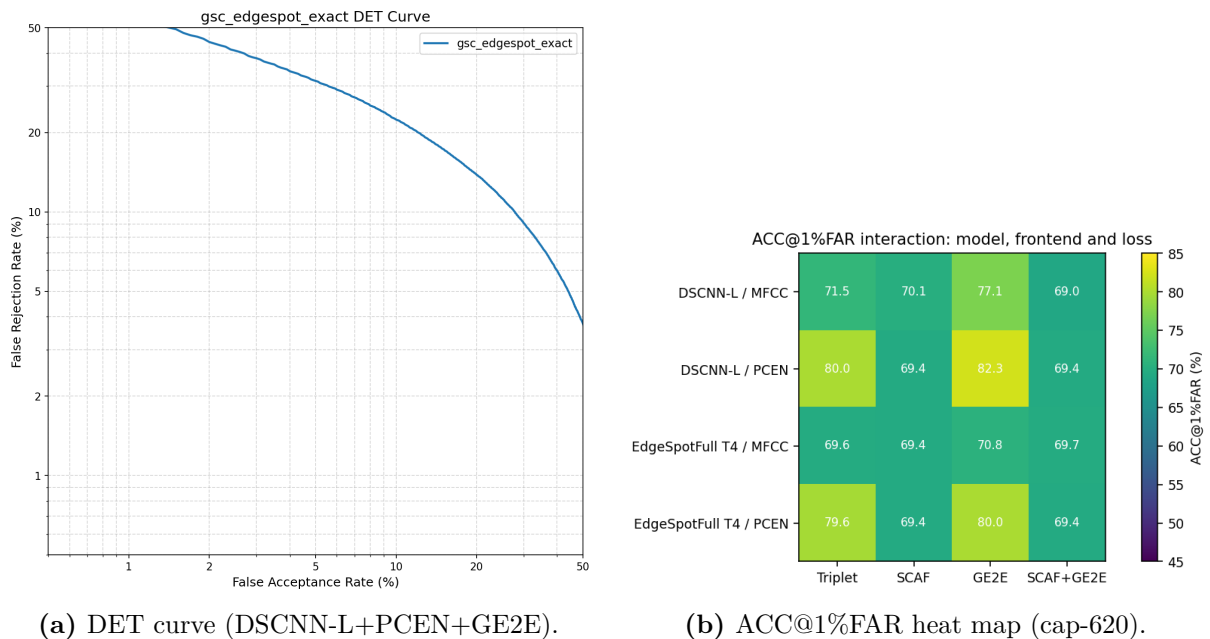


Figure 4.6: Headline operating-point views. (a) the detection-error-tradeoff curve of the best overall system; (b) the ACC@1%FAR heat map over the cap-620 configurations, where the PCEN+GE2E cell is the brightest at both backbones. *Project-generated result figures.*

Practical implication. The consolidated table gives a deployer two clear options depending on hardware constraints: when accuracy is the priority and resources allow,

use DSCNN-L+PCEN+GE2E; when memory/battery are limited, the three-times-smaller EdgeSpot-Full is only about 3.5 points behind — an acceptable trade-off for an edge device. More importantly, the same encoder serves unlimited downstream keyword sets without retraining, so this one-time cost is amortized across every application.

4.6 Inference on long audio

To verify that the system behaves end-to-end — not only on isolated test clips — we enroll prototypes for 17 keywords (10 shots each from GSC) and run nearest-prototype inference over a real, continuously spoken multi-word recording.

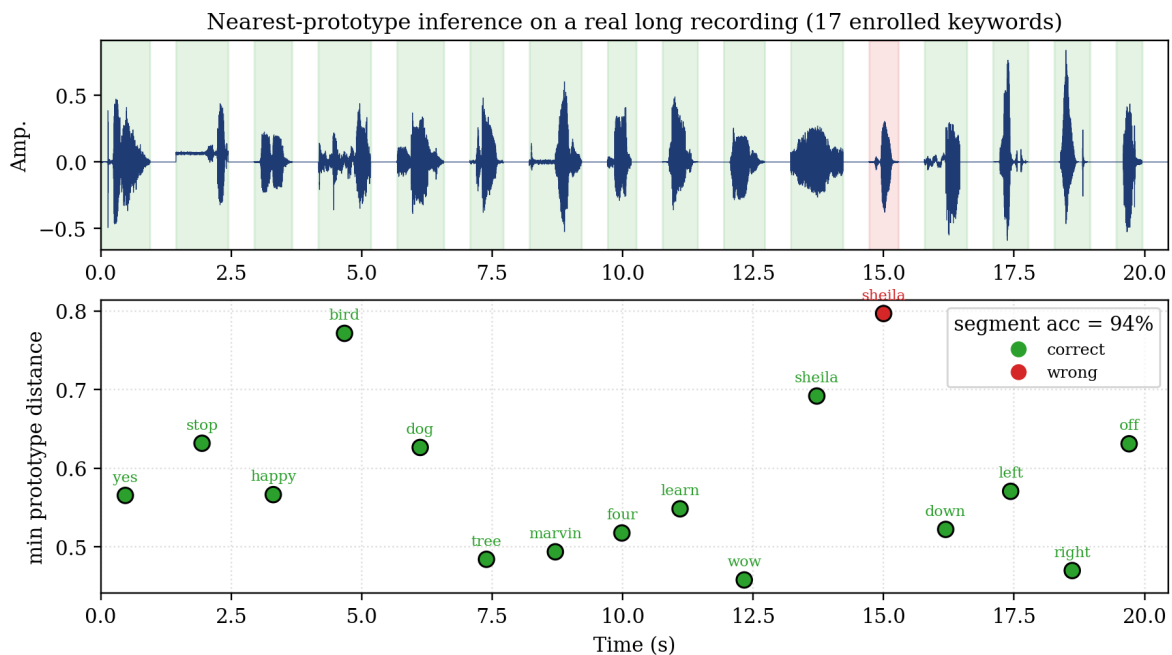


Figure 4.7: Nearest-prototype inference on a real long recording. Top: waveform with per-segment correct (green) / wrong (red) shading. Bottom: minimum prototype distance and predicted keyword per segment.

Figure 4.7 overlays the predicted keyword and its prototype distance on the waveform for the first 16 spoken segments. The model correctly identifies 15/16 segments ($\approx 94\%$); the single error (“sheila”, a long, rare name) also has the largest prototype distance, exactly the signal the open-set threshold uses to flag low-confidence detections.

Chapter 5

Conclusion

This chapter closes the thesis by reviewing the whole arc: from the problem posed in Chapter 1, through the method in Chapter 3, to the experimental evidence in Chapter 4. We summarize the current status of the system, state its strengths and weaknesses honestly, and outline the next directions that follow directly from the limitations observed.

5.1 Current status

This thesis set out to build a keyword-spotting system that escapes the two structural limits of classical wake-word engines — a frozen vocabulary and the inability to abstain — and to understand, rather than merely report, what makes such a system work as the training vocabulary grows. We delivered a complete, reproducible, dependency-light pipeline: two compact encoders (DSCNN-L at ≈ 0.41 M and EdgeSpot-Full at ≈ 0.13 M parameters), three feature front-ends, four metric-learning objectives, four open-set rejection heads, a CPU streaming engine, and an offline web demonstrator. Every system is trained on MSWC English and evaluated cross-corpus on GSC v2 under a single strict EdgeSpot-exact 10-shot protocol with 100 randomized runs, so the numbers compare fairly.

The headline outcome is that few-shot open-set KWS is viable and even strong at true vocabulary scale: training once on 37,387 words, the best configuration (DSCNN-L + PCEN + GE2E) reaches 86.36% open-set accuracy at a strict 1% false-accept rate and 95.21 AUC on a corpus it was never trained on, while a three-times-smaller edge model stays within a few points. A user can define an arbitrary keyword set from ten examples per word, with no retraining, and the system can decline to answer on unknown speech and silence.

5.2 What is good, what is weak

What works. Three findings are robust and transferable. (1) The trainable PCEN front-end is the most reliable single improvement, and it helps the smaller encoder most — a useful lever precisely where the parameter budget is tightest. (2) The centroid GE2E objective, which optimizes the same enroll-then-match computation used at deployment, consistently beats both triplet and classification objectives for open-set generalization. (3) A frozen Wav2Vec 2.0 teacher distilled with a simple cosine loss lets the compact model reach GE2E-level accuracy with roughly half the data, which matters when data or compute are limited.

What is weak / what we learned the hard way. The clearest negative result is the collapse of the sub-center ArcFace (SCAF) classification objective at 10^4 -class scale: a loss that is excellent on tens to hundreds of classes drives the embedding to a trivial solution on tens of thousands of classes unless its weight is cut by four orders of magnitude. This is an important cautionary lesson for anyone scaling angular-margin losses to large label spaces. Two additional limitations bound the present claims. First, the ACC@FAR/FRR@FAR operating points are read from the evaluation ROC (an oracle threshold) rather than transferred from an independent calibration split; we mitigate this by always co-reporting threshold-free AUC/EER/F1, but a calibrated, deployment-time threshold study is still owed. Second, enrollment is fixed at $k=10$ shots and the streaming latency figures are design budgets, not measured field benchmarks; there is also no quantitative denoising ablation.

5.3 Future work

Four directions follow directly from the limitations above. (1) *k-shot sensitivity*: sweep $k \in \{1, 3, 5, 10\}$ to chart the accuracy-vs-enrollment-effort curve, which is the metric end users actually feel. (2) *Calibrated thresholds*: learn or transfer the open-set threshold from a held-out split and report deployment-realistic ACC@FAR, closing the oracle gap. (3) *On-device benchmark*: measure end-to-end latency, false-alarms-per-hour and miss-rate on target hardware, plus an SNR-sweep denoising ablation, to validate the design budgets. (4) *Stabilizing classification at scale*: margin/scale warm-up, curriculum, or sub-center pruning to recover the discriminative power of angular-margin losses without the collapse — potentially combining SCAF’s tight clusters with GE2E’s deployment-matched objective at full vocabulary.

In summary, the thesis demonstrates a deployable, retraining-free, open-set keyword spotter and, just as importantly, a clear empirical map of which design choices help, where they saturate, and where they fail as the problem scales.

Bibliography

- [1] Y. Zhang, N. Suda, L. Lai, and V. Chandra, “Hello edge: Keyword spotting on microcontrollers,” *arXiv:1711.07128*, 2017.
- [2] B. Kim, S. Chang, J. Lee, and D. Sung, “Broadcasted residual learning for efficient keyword spotting,” *Interspeech*, 2021.
- [3] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” *NeurIPS*, 2017.
- [4] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” *CVPR*, 2015.
- [5] L. Wan, Q. Wang, A. Papir, and I. L. Moreno, “Generalized end-to-end loss for speaker verification,” *ICASSP*, 2018.
- [6] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” *CVPR*, 2019.
- [7] J. Deng, J. Guo, T. Liu, M. Gong, and S. Zafeiriou, “Sub-center ArcFace: Boosting face recognition by large-scale noisy web faces,” *ECCV*, 2020.
- [8] A. Bendale and T. E. Boulton, “Towards open set deep networks,” *CVPR*, 2016.
- [9] W. Liu, X. Wang, J. Owens, and Y. Li, “Energy-based out-of-distribution detection,” *NeurIPS*, 2020.
- [10] Y. Wang, P. Getreuer, T. Hughes, R. F. Lyon, and R. A. Saurous, “Trainable frontend for robust and far-field keyword spotting,” *ICASSP*, 2017.
- [11] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” *NeurIPS*, 2020.
- [12] M. Mazumder *et al.*, “Multilingual Spoken Words Corpus,” *NeurIPS Datasets and Benchmarks*, 2021.
- [13] P. Warden, “Speech Commands: A dataset for limited-vocabulary speech recognition,” *arXiv:1804.03209*, 2018.